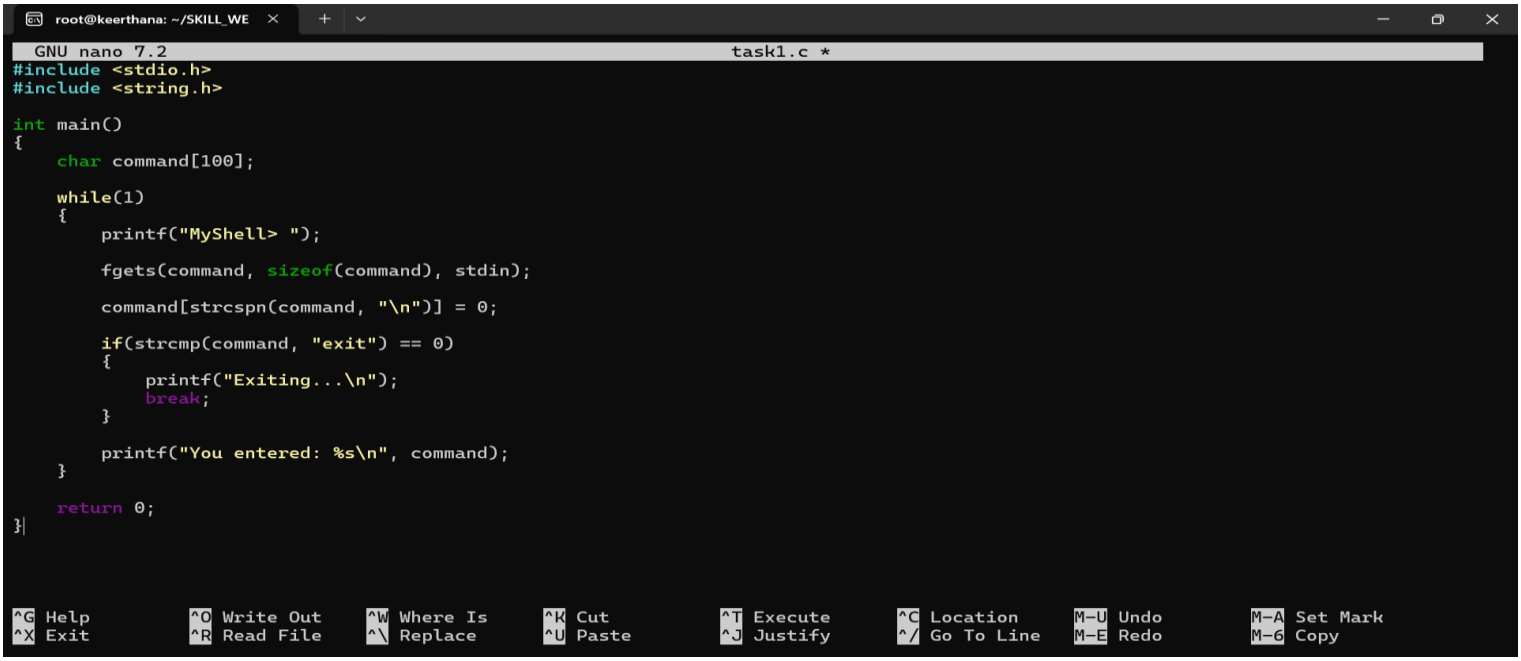
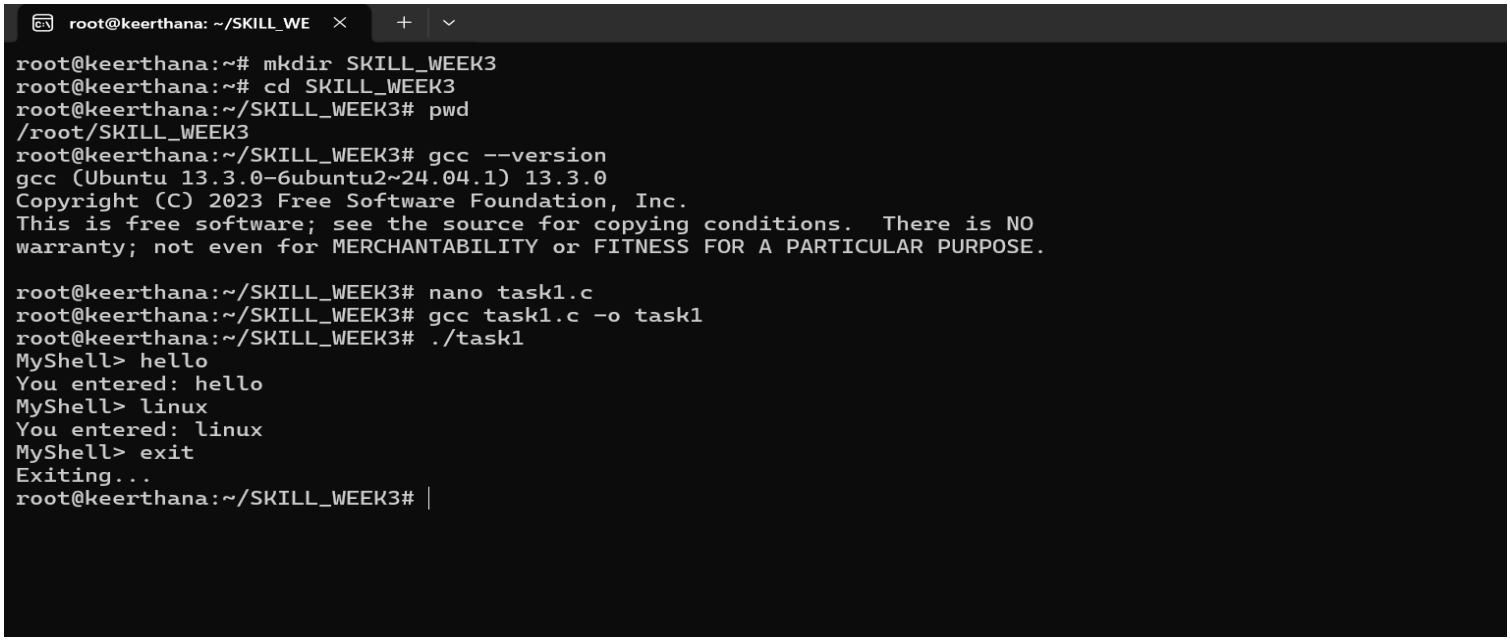
Task 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop





Task 2: Using the following system calls, copy the content from File1.txt to File2.txt.

1. **Open()**
2. **Read()**
3. **Write()**
4. **Close()**

open()

syntax: open(" filename.txt", modes, permissons)

Example: open("First_Prgm.txt", O_CREAT | O_WRONLY, 0644);

Table 1: Modes of Open system call

Flag	**Purpose**
O_RDONLY	Open for read only
O_WRONLY	Open for write only
O_RDWR	Open for both read and write
O_CREAT	Create file if it does not exist
O_TRUNC	Truncate (empty) an existing file
O_APPEND	Append data to the end of the file
O_EXCL	Fail if file already exists (used with O_CREAT)

read()

Syntax: read(int fd, void *buf, size_t count);

Example: read(fd, buffer, 100)

#include <stdio.h>

```
#include <unistd.h>
int main()
{
   char buffer[100];
   int n;
```

```
    printf("Enter some text: ");
    n = read(0, buffer, sizeof(buffer) - 1);
    buffer[n] = '\0';   // Null-terminate the string
    printf("You entered: %s", buffer);
    return 0;
}
```

write()

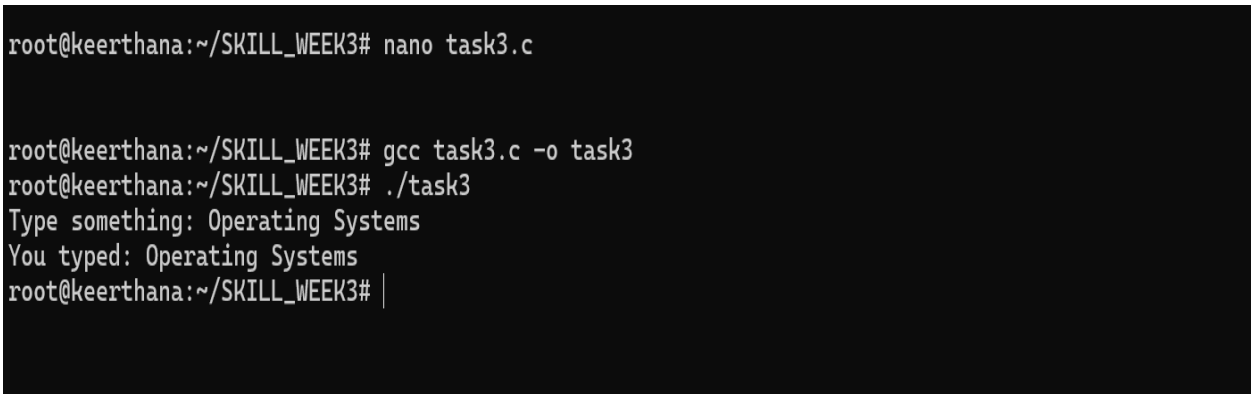
Syntax: write(int fd, const void *buf, size_t count);

Example: write(fd, message, 11);

```
#include <unistd.h>

int main()

{
    write(1, "Hello World\n", 12);
    return 0;
}
```


```
root@keerthana:~/SKILL_WEEK3# nano File1.txt
root@keerthana:~/SKILL_WEEK3# nano task2.c


root@keerthana:~/SKILL_WEEK3# gcc task2.c -o task2
root@keerthana:~/SKILL_WEEK3# ./task2
File copied successfully.
root@keerthana:~/SKILL_WEEK3# cat File2.txt
Welcome to Ubuntu
Operating System Lab

root@keerthana:~/SKILL_WEEK3#
```

Task 3: To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.



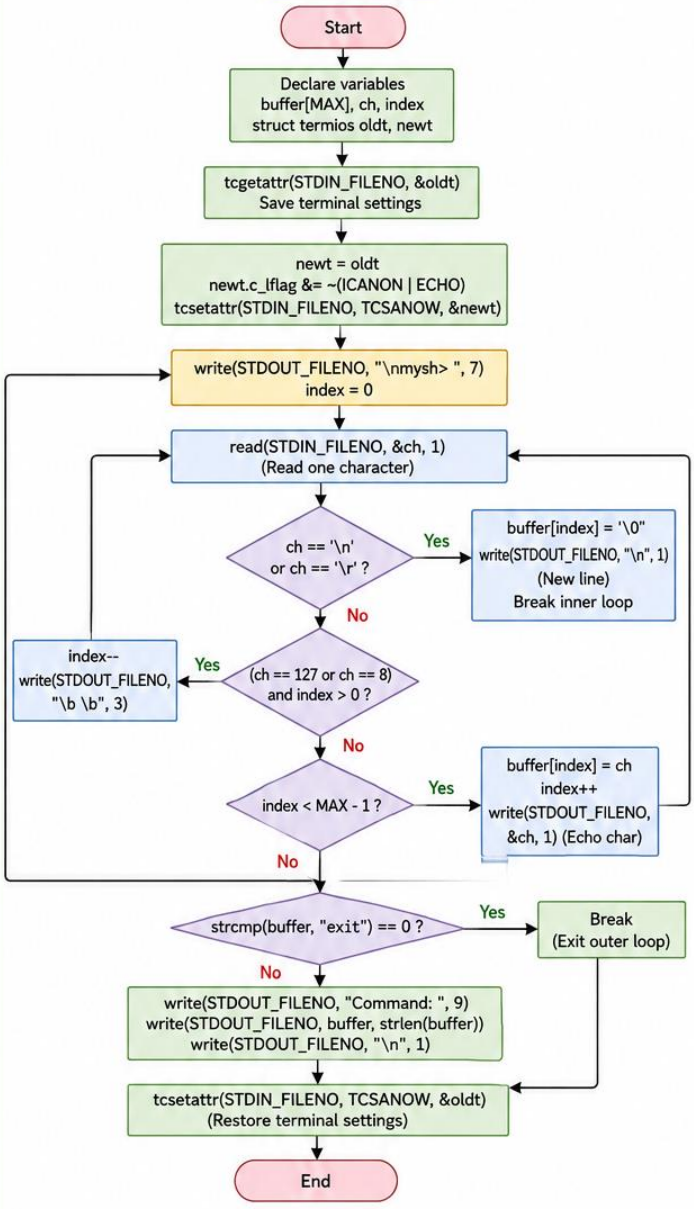
ALGORITHM & FLOWCHART

Keyboard Input Handling Using System Calls (read, write, termios)

ALGORITHM

1. Start.
2. Declare variables:
   - buffer[MAX]
   - char ch
   - int index
   - struct termios oldt, newt
3. Save the current terminal settings using tcgetattr(STDIN_FILENO, &oldt).
4. Copy the terminal settings: newt = oldt
5. Disable canonical mode and echo: newt.c_lflag &= ~(ICANON | ECHO)
6. Apply the new terminal settings using tcsetattr(STDIN_FILENO, TCSANOW, &newt).
7. Repeat forever
   (a) Display the prompt using write(): "mysh> "
   (b) Set index = 0.
   (c) Read characters one by one using read().
   (d) If the character is Enter ('\n' or '\r'):
      - buffer[index] = '\0'
      - Print new line.
      - Exit the inner loop.
   (e) Else if the character is Backspace (ASCII 8 or 127) and index > 0:
      - index--
      - Print "\b \b" to remove last character.
   (f) Else if index < MAX - 1:
      - Store the character in buffer.
      - index++
      - Echo the character using write().
   (g) After exiting the inner loop, compare the command in buffer.
      - If command is "exit", break the outer loop.
      - Otherwise, print "Command: " followed by the command.
8. Restore the original terminal settings using tcsetattr(STDIN_FILENO, TCSANOW, &oldt).
9. End.

FLOWCHART



SYSTEM CALLS / FUNCTIONS USED

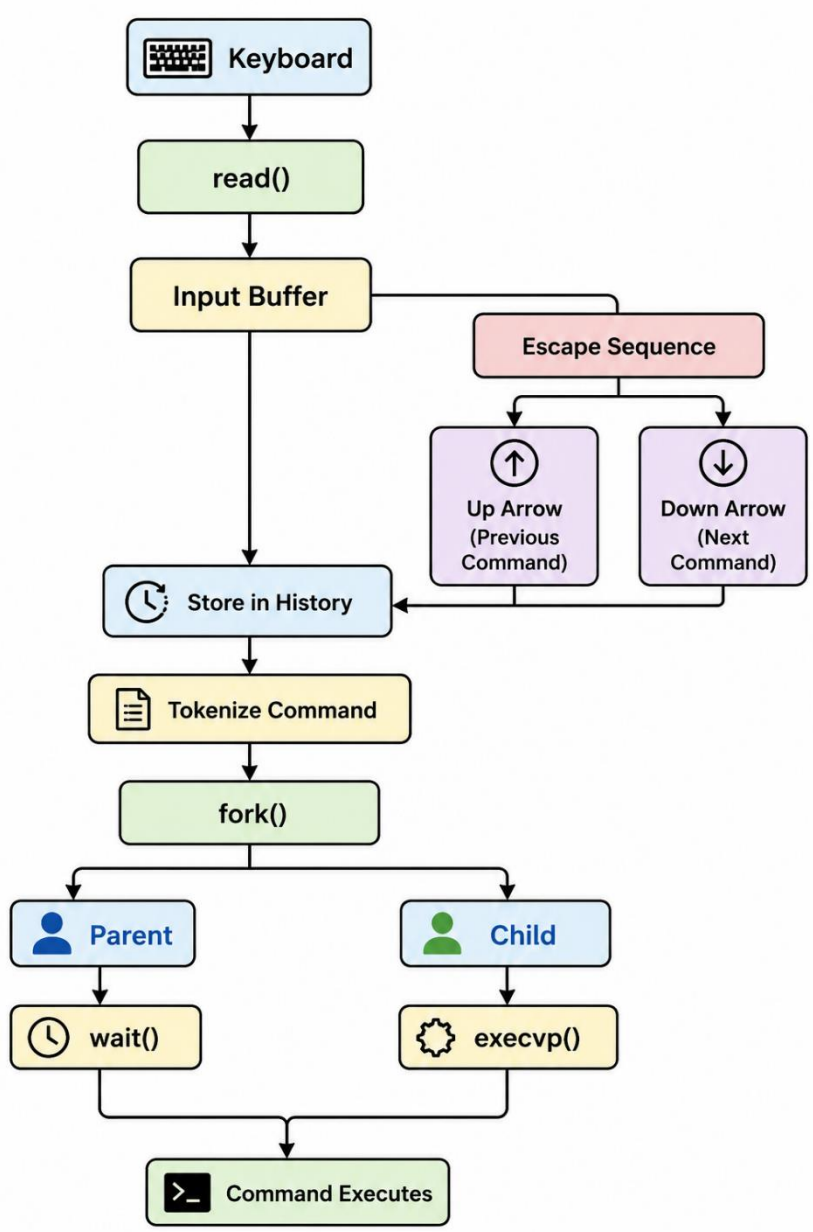
Function / System Call	Purpose
read()	Reads one character from keyboard (stdin).
write()	Writes output to terminal (stdout).
tcgetattr()	Gets current terminal attributes.
tcsetattr()	Sets terminal attributes (raw / canonical mode).
strcmp()	Compares strings (used to check "exit").

TERMINAL MODE

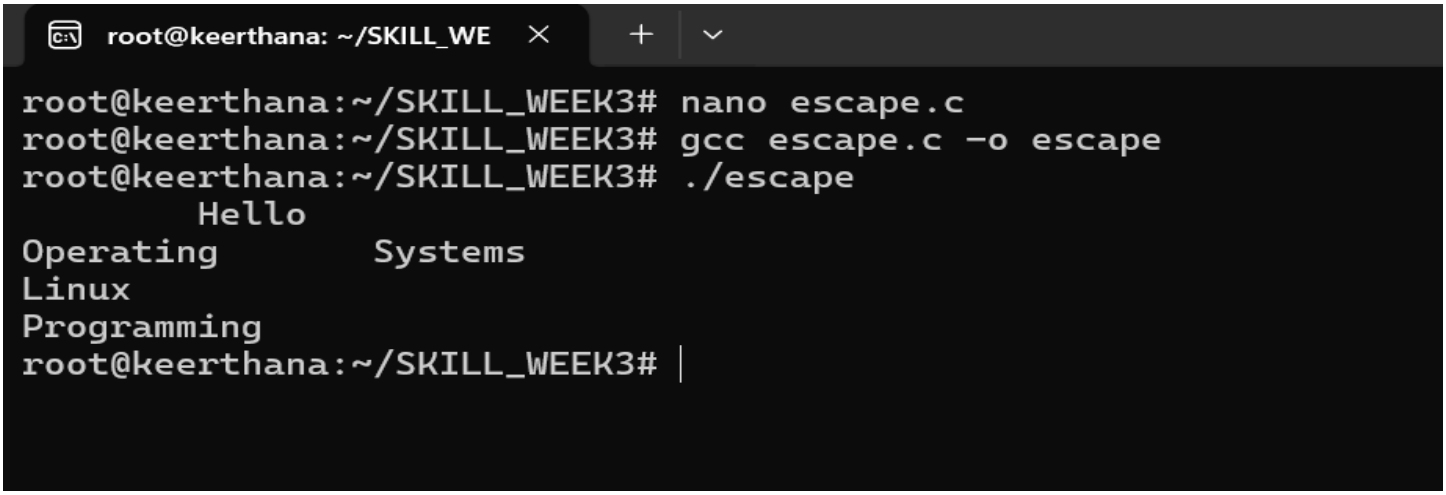
- Canonical mode (default): Input is line buffered. Backspace is handled by terminal driver.
- Non-canonical (raw) mode: Input is available character by character; program handles Backspace, Enter, etc.
- We disable ICANON (canonical) and ECHO to read characters immediately and handle them in the program.

This program uses system calls (read, write) and terminal control (termios) to capture keyboard input, handle backspace, process enter key, manage input buffer and support multi-character commands.

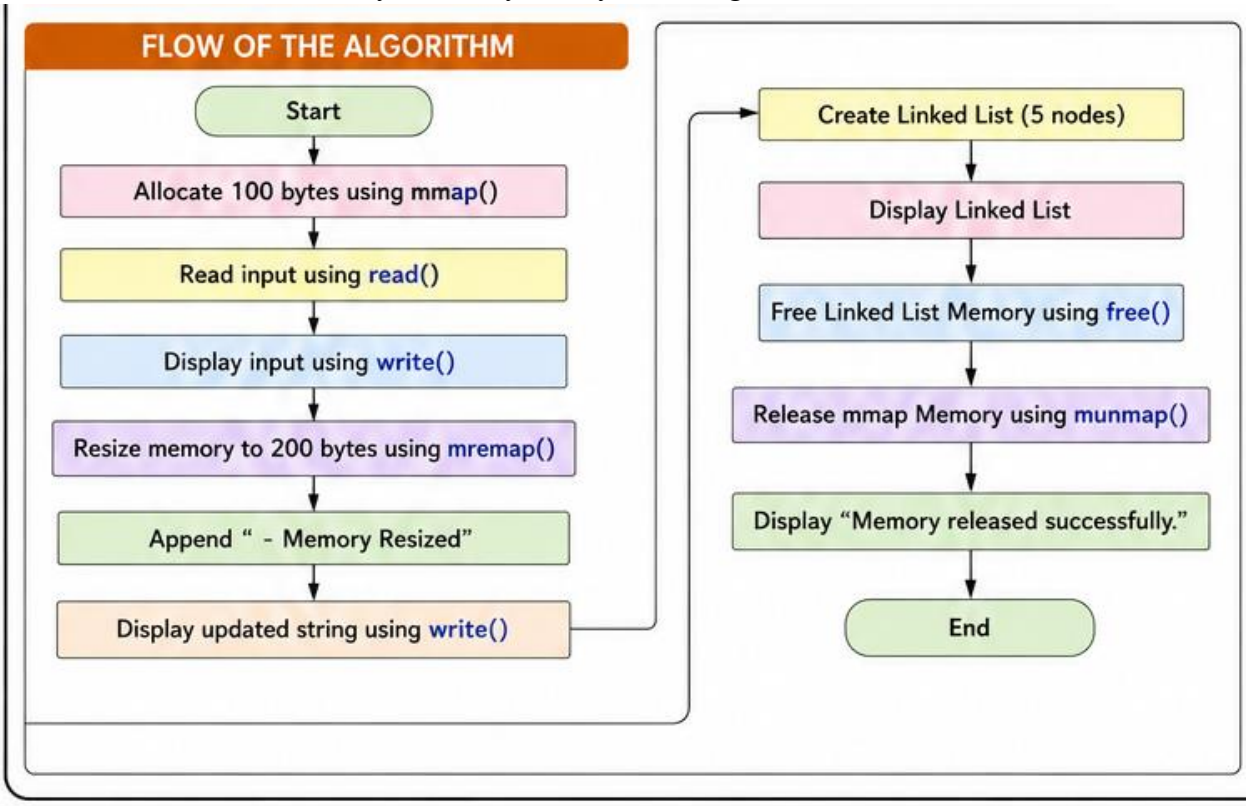
Task 4: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality



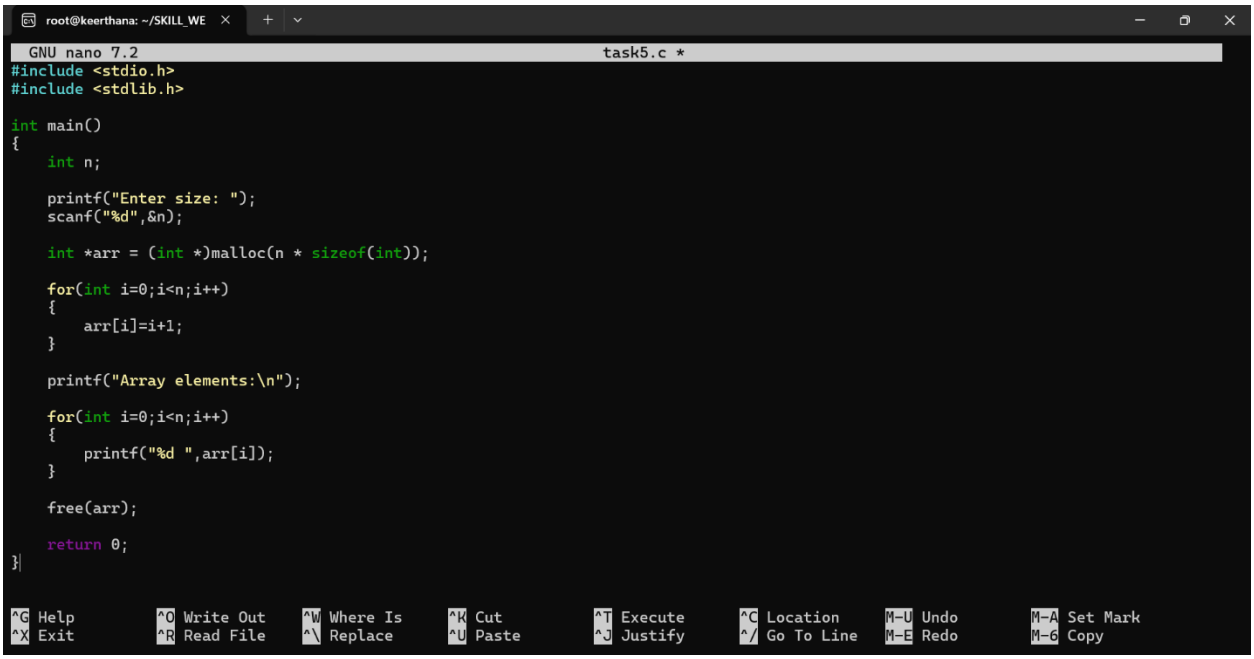

root@keerthana: ~/SKILL_WE
GNU nano 7.2 escape.c *
#include <stdio.h>
int main()
{
printf("\tHello\n");
printf("Operating\tSystems\n");
printf("Linux\nProgramming\n");
return 0;
}
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy

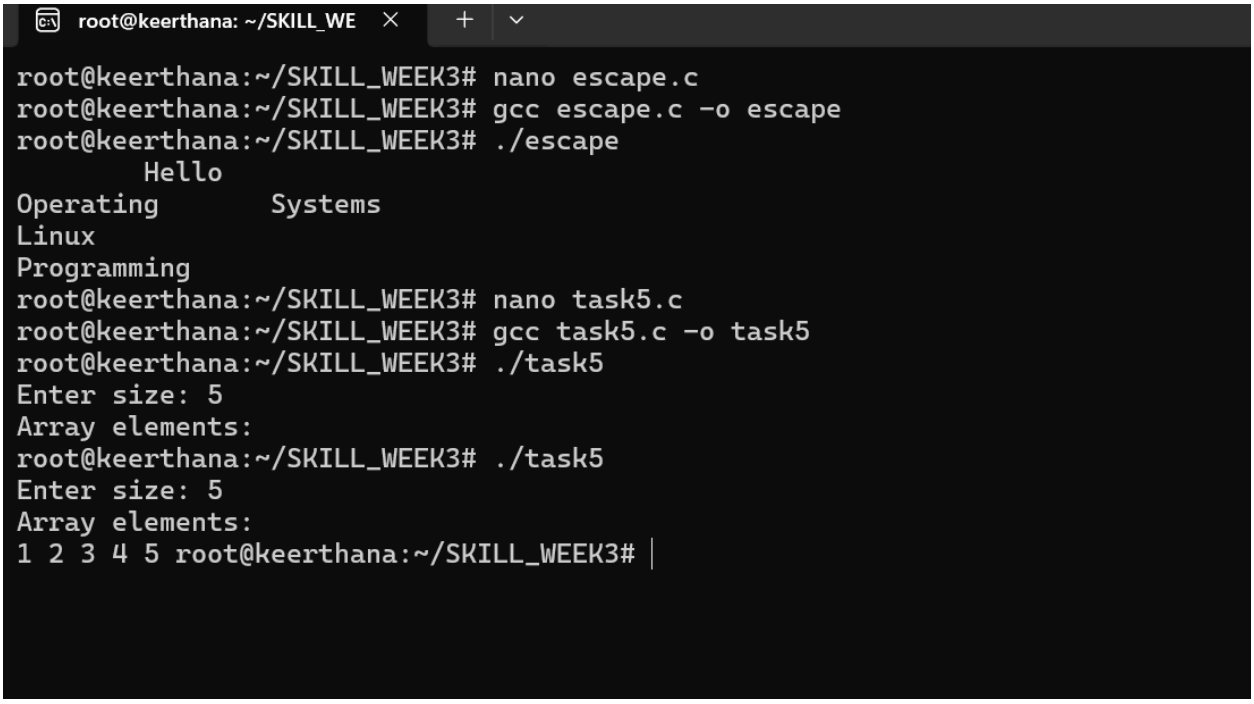

root@keerthana: ~/SKILL_WE
root@keerthana:~/SKILL_WEEK3# nano escape.c
root@keerthana:~/SKILL_WEEK3# gcc escape.c -o escape
root@keerthana:~/SKILL_WEEK3# ./escape
Hello
Operating Systems
Linux
Programming
root@keerthana:~/SKILL_WEEK3#

Task 5: To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind.



Validate the program with valgrind tool


root@keerthana: ~/SKILL_WE
GNU nano 7.2 task5.c *
#include <stdio.h>
#include <stdlib.h>
int main()
{
int n;
printf("Enter size: ");
scanf("%d",&n);
int *arr = (int *)malloc(n * sizeof(int));
for(int i=0;i<n;i++)
{
arr[i]=i+1;
}
printf("Array elements:\n");
for(int i=0;i<n;i++)
{
printf("%d ",arr[i]);
}
free(arr);
return 0;
}
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy


root@keerthana: ~/SKILL_WE
root@keerthana:~/SKILL_WEEK3# nano escape.c
root@keerthana:~/SKILL_WEEK3# gcc escape.c -o escape
root@keerthana:~/SKILL_WEEK3# ./escape
Hello
Operating Systems
Linux
Programming
root@keerthana:~/SKILL_WEEK3# nano task5.c
root@keerthana:~/SKILL_WEEK3# gcc task5.c -o task5
root@keerthana:~/SKILL_WEEK3# ./task5
Enter size: 5
Array elements:
root@keerthana:~/SKILL_WEEK3# ./task5
Enter size: 5
Array elements:
1 2 3 4 5 root@keerthana:~/SKILL_WEEK3#